

导购程序工作原理

TikTok TechJam 2026 · 赛道四 · 用大白话讲清楚我们的参赛程序 · 公开测试得分 0.9748（实际满分上限 0.9922）

1 这个游戏怎么玩

有一位模拟的顾客，心里想买一件商品。这件商品藏在一个 50,000 件的服饰鞋包目录里，顾客不会说出它的名字。

我们最多和顾客聊十轮。每一轮可以做两件事：问一个问题，以及展示一份最多十件商品的名单。只要那件商品出现在名单里，这一局立刻结束。出现得越早、排得越靠前，得分越高。始终没出现，这一局零分。

评委会用我们的程序跑 800 局这样的游戏，全程没有真人。我们交的东西就是一个 Python 类，只有两个方法：一个开局，一个逐轮应答。

2 关键事实：顾客在念答案

顾客不是人，是一段程序，而且源代码是公开的。开局之前，它会翻开那件商品自己的目录页，从里面抄出最多四条短句——材质、卖点、价格。这四条就是它的**线索**。顾客说的每一句话，都是用这些线索拼出来的。

所以这个游戏的本质是：**顾客拿着答案在给我们念题——但一次只念一两条，而且要我们开口问才念。**我们的整个设计都从这句话推出来：把线索收集齐，找出唯一全部符合的商品，用尽量少的轮数。

还有一条规则很要紧：顾客念过的线索绝不重复。念过一条，就永远不会再说。所以听到的每一条，都必须留住。

3 流水线

顾客的每句话都经过五个步骤，然后我们的回复发回去，进入下一轮。

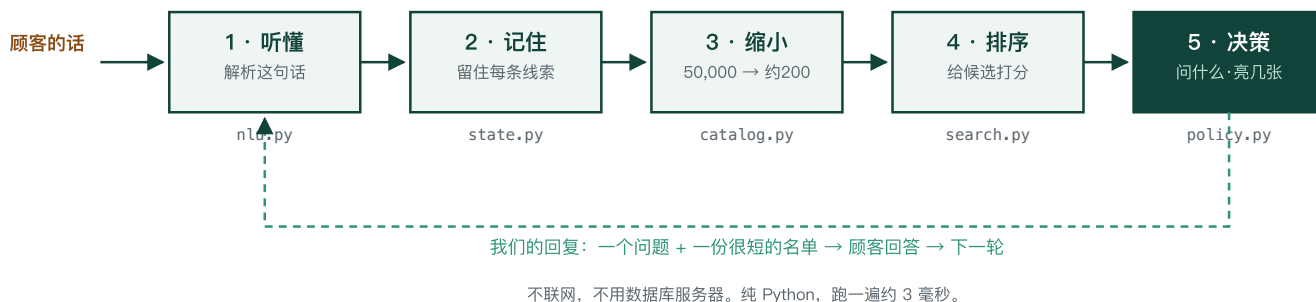


图 1：一轮对话。这个循环最多转十次。

4 五个步骤，以及每一步为什么这样设计

第一步——听懂（三层解析）

先用固定句式去匹配。公开的顾客程序只会用八种模板造句，所以绝大多数话一次就解析成功。如果模板对不上——评委可能会把句子改写——第二层就改抓意思：“想找……”、“必须有……”、“别管我刚才说的……”这类关键短语。第三层可以调用大模型帮忙，但默认关闭。程序不需要它，也完全不需要联网。

第二步——记住（把旧线索调小声，绝不扔掉）

每条线索都存进一个格子。顾客有时会说：“算了，别管我刚才的偏好。”直觉上应该把旧格子清空。我们实测过：清空是双重错误。第一，顾客念过的线索永不重复，删掉就永远没有了。第二，那条“被放弃”的偏好，其实仍然是那件商品的真实特征——顾客换的是说法，不是目标。所以我们保留旧线索，只是把权重调低。就这一个决定，救回了早期原型输掉的仅有两局。

第三步——缩小（类目池）

顾客第一句话会报出类目：“我想找皮带配饰。”我们预先建好一张索引：类目 → 该类目下的所有商品。就凭这一句话，候选从 50,000 件缩到中位数 184 件。在全部 200 局公开测试里，那件藏着的商品每一次都在这个池子里——200 局，200 次。

第四步——排序（先看证据，再看人气）

池子里每件商品打一个分。它是不是一字不差地含有某条线索？含有线索的全部词？部分词？罕见词比常见词值钱，所以“五芒星”比“项链”有分量。平时比人气，而人气在这里绝不是小事：藏着的商品来自真实购买，真实购买集中在热门货。藏品的评价数中位数是 6,846 条，而整个目录的中位数只有 12 条。

第五步——决策（一次只押一件）

每轮做两个决定：问什么，亮几张牌。

问什么。永远问开放式问题——“还有什么别的要求吗？”顾客手里有四条线索，每问一次最多交出两条，问两次就全部到手。问窄了（比如只问颜色）可能落空，开放式问题不会。

亮几张。一张。这是最反直觉的一步，讲一下算术。一次亮十件，等于十件一起下注：如果目标排在第七，就按“猜中第七名”付钱——赔率很差。一轮只亮一件，等于一连串单注：只要押中，就是按第一名付钱。第一名和第七名差 0.26 分，而多等一轮只扣 0.02 分。耐心的赔率大约是十五比一。所以我们每轮只亮当前最有把握的一件，亮过的不再重复；只在最后两轮撒开成完整名单，作为保险。

从“每轮亮十件”改成“一次押一件”，总分从 0.901 升到 0.975；命中名次的平均成绩从 0.71 升到 0.995——1.0 的意思是**每一局都押中在第一名**。

5 三局真实对话

以下是真实记录，全部用官方评测程序原样重放。

第一局——目标明确的顾客（藏品：一条五芒星吊坠项链）

顾客	我想找项链首饰。硬性要求：材质为合金。
程序	亮出当前最佳的一件（押错了——合金项链有几千条），然后问：还有别的要求吗？
顾客	我在意的是：三相月五芒星图案；……
程序	亮出那条吊坠。命中——第 2 轮，第 1 名。罕见词立了功。

第一条线索几乎没用，第二条却是指纹。这就是“先问再猜”胜过“闷头猜”的原因。

第二局——中途改主意的顾客（藏品：一条真皮皮带）

顾客	我想找皮带配饰。搭扣式的。
程序	亮出最佳一件，照常问开放式问题。
顾客	我在意的是：皮革；100% 真皮。
程序	亮出了正确的那条皮带……但有条隐藏规则：顾客改主意之前的命中一律不算数。这一局继续。
顾客	算了，别管我刚才的偏好。我要的是：皮革。
程序	留住全部线索（权重调低），清空出牌记录，重新排序。
顾客	我在意的是：进口；搭扣式。
程序	再次亮出同一条皮带。命中——第 4 轮，第 1 名。这次算数了。

如果在顾客改主意时清空了旧线索，手里就只剩“皮革”两个字——而皮革皮带有几千条。留住线索，才赢下这一局。

第三局——随便逛逛的顾客（藏品：一条篮球短裤）

顾客	我在看男式篮球用品，还在随便逛。
程序	亮出该类目里人气最高的一件。命中——第 1 轮，第 1 名。

一条线索都没有——类目加上人气先验就够了。这种局比想象中多：6,846 比 12 的人气差距就体现在这里。

6 得分

官方总分由三部分组成：找没找到（权重 0.50）、排得多靠前（0.30）、用了几轮（0.20）。完美发挥也只能拿 0.9922 而不是 1.0——有些轮数是规则强制的。

测量（官方评测程序，未做任何修改）	得分
全部 200 局公开测试	0.9748
50 局留出测试（调参时从未使用）	0.9648
把顾客每句话都改写（抗改写测试）	0.9566
把一半线索词换成完全陌生的词	0.9248
官方基线程序（对照）	0.1067

找到：200 局全中。命中名次：200 局里 198 局排第一。平均命中轮数：2.19。

7 还有什么可能出问题

我们的程序是对着一个源代码公开的模拟器调出来的。评委的 800 局私有测试用同一套代码，但换了商品，句子也可能被改写。上表的两行抗性测试量的正是这件事，跌幅都不大——因为设计依赖的是改写改不掉的东西：类目、罕见词、人气。另外，200 局测试自带约 ± 0.03 的统计噪声。所以对私有测试的诚实预期是：**0.95 左右，上下 0.03**——报区间，不报点。

代码仓库：`agent.py`（入口）· `src/`（五个步骤）· `tools/`（评测入口、两套抗性测试）· `tests/`。零依赖；Python 3.9+；完全离线。本文所有数字都可用 README 中的单条命令复现。